MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

DBS401 COURSE PROJECT REPORT

Developing a PostgreSQL Database Honeypot with AI-Based Attacker Behavior Analysis

Group 3

Phan Thanh Thảo	SE193364	Lecturer:	Hồ Hải
Trần Thị Minh Anh	SE190902	Course:	DBS401
Vũ Trung Hiếu	SE192673		

Ho Chi Minh City

October 2026

Abstract

Internet-exposed database systems are attractive targets because they hold valuable information and expose direct query interfaces. Database honeypots can provide a controlled environment for observing these interactions, yet the usefulness of collected traces depends on what the logging configuration records and whether separate queries can be reconstructed as coherent attack sessions. This project proposes a local PostgreSQL honeypot with synthetic sensitive-looking data and a controlled, interactive query environment to study logging coverage and automated behavior analysis. Parameterized attack and benign scenarios will generate labeled query sequences with explicit dependencies, while four predefined logging profiles will be parsed into a common event schema. The study will first compare action-level recall, event-field completeness, and session reconstruction across inactivity thresholds. It will then compare a rule-based classifier with a locally run few-shot large language model on query windows containing the current query and selected preceding queries. Classification will be evaluated using macro-averaged F1 and per-class results; explanations will be evaluated against dependency-derived evidence using precision, recall, and F1. Finally, both methods will be evaluated without modification on isolated attack-technique families withheld during development. The planned outcomes are an empirical assessment of logging trade-offs, a reproducible query-level evaluation pipeline, and evidence about classification and explanation robustness under technique variation. Because the experiments have not yet been conducted, this abstract describes the project design and intended evaluation rather than empirical findings.

Keywords— PostgreSQL honeypot, database auditing, attack-session reconstruction, behavioral analysis, few-shot learning, evidence-based classification, out-of-distribution generalization.

Contents

Abstract **i**

1 Introduction **1**

1.1 Background 1

1.2 Problem Statement 2

1.3 Research Questions and Objectives 3

1.3.1 RQ1: Logging and Session Reconstruction 3

1.3.2 RQ2: Classification and Evidence 3

1.3.3 RQ3: Generalization to Unseen Technique Families 4

1.4 Significance of the Project 4

1.5 Scope and Limitations 5

1.6 Report Organization 7

2 Literature Review **8**

2.1 Scope of the Review 8

2.2 Database Honeypots and Attacker Behavior 8

2.3 Audit Logging, Session Analysis, and Attack Reconstruction 9

2.4 LLM-Based Interpretation and Technique Mapping 11

2.5 Classical Machine Learning and LLMs on Structured Security Data 12

2.6 Generalization to Unseen Attack Behavior 13

2.7 Honeypot Realism, Synthetic Decoy Data, and Evaluation 13

2.8 Summary of the Literature Review 14

2.9 Contribution of Research 15

3 Methodology **16**

3.1 Research Design 16

3.2 Honeypot Environment and Logging Profiles 16
3.3 Attack Taxonomy and Scenario Generation 17
3.3.1 Behavior labels 17
3.3.2 Parameterized scenario generator 18
3.4 Data Collection 19
3.4.1 RQ1 data 19
3.4.2 RQ2 and RQ3 data 19
3.5 Data Splitting 20
3.6 Analysis Techniques 20
3.6.1 RQ1: coverage and session reconstruction 20
3.6.2 Rule-based classifier 21
3.6.3 LLM few-shot classifier 21
3.6.4 Evaluation metrics 23
3.6.5 Effect of context length 23
3.6.6 Uncertainty and comparison 24
3.6.7 RQ3: performance under unseen families 24
3.7 Limitations of the Methodology 25
3.8 Reproducibility and Ethical Considerations 25

Chapter 1

Introduction

1.1 Background

Database management systems are part of the critical data layer of modern applications. When a database service is exposed through a public network, an attacker may probe its identity and configuration, attempt to authenticate, issue queries to discover or collect data, and try to alter database state or use database capabilities to reach other resources. Unlike a single isolated query, these actions can form a sequence in which later steps depend on information or access obtained earlier. Understanding such behavior therefore requires both reliable event capture and analysis that preserves the relationships among events.

A database honeypot provides a controlled way to observe activity directed at a decoy service. Song, Smaragdakis, and Griffioen deployed 278 honeypots over 20 days, including low-interaction emulations of PostgreSQL and other database systems, and reported persistent brute-force attempts together with medium- and high-interaction exploitation that manipulated data, copied and deleted it, or used the DBMS to reach the underlying system [1]. Earlier work demonstrated how a database honeypot can be built on open-source MySQL and used together with honeytokens for deception [2]. These studies establish the value of database-focused deception, while also motivating careful measurement of what a particular database deployment can observe.

The raw trace is the foundation for downstream analysis. A logging profile that captures statements but omits relevant connection, error, or parameter information may leave gaps in the account of an interaction. Conversely, highly detailed logging can increase storage use and query-processing overhead, and may expose sensitive values if real data is used. Even when individual events are recorded, analysts still need to determine which events belong to the same logical session and which queries provide evidence for a particular attack stage. The logging

design is therefore not merely an operational setting; it directly affects the completeness and reliability of subsequent behavior analysis.

Recent work has also explored large language models for interpreting security alerts and mapping low-level observations to higher-level attack behavior. Schärmer et al. evaluate ChatGPT and Gemini for classifying IDS alerts and report that carefully chosen few-shot examples and system context support more effective classification, while noting limitations related to cost, reliability, and sensitive-data exposure [3]. Meng et al. present RHINO, which maps network logs to adversarial tactics and techniques; they argue that single-step LLM classification often hallucinates technique names and produces decontextualized mappings, and they propose guided reasoning as an alternative [4]. These findings motivate testing contextual classification and evidence selection on database query traces, rather than assuming that a model label alone is a reliable explanation.

1.2 Problem Statement

Despite the relevance of honeypots and language-model-assisted security analysis, three practical questions remain for the proposed PostgreSQL setting. First, logging profiles may differ in their ability to capture the actions needed to identify seven project-defined behavior classes: Discovery, Credential Access, Privilege Escalation, Collection, Exfiltration, Impact, and Noise. Six of these classes describe attack stages and one describes benign activity. The project must quantify these differences and determine whether the resulting records contain the fields required for later analysis. Second, captured log lines may not directly identify a complete attack session. Sessionization based on an inactivity threshold can merge unrelated activity or split a real multi-step sequence, and its accuracy must be measured against known scenario ground truth. Third, query-level classification is difficult when the meaning of a query depends on earlier events. Rule-based methods may be transparent but brittle to variations; a few-shot language model may use context more flexibly but can produce unsupported labels or evidence identifiers. Neither approach should be considered reliable without measuring both classification and evidence quality.

A further challenge is generalization. A method can perform well on variations that resemble its development scenarios while failing on a technique family that was not used to write its rules or examples. To assess this distinction, the evaluation needs a controlled split by scenario family, with held-out families kept isolated from method development. It must also compare

not only predicted labels but the query evidence used to justify them. Without this separation and evidence-level evaluation, apparent performance may reflect leakage or familiar patterns rather than the ability to analyze unseen behavior.

Accordingly, this project will develop a reproducible, local evaluation pipeline that connects PostgreSQL logging, session reconstruction, query classification, and evidence assessment. The project focuses on measuring the end-to-end suitability of captured interaction traces for behavior analysis, not on claiming that synthetic scenarios represent the full diversity of real-world attackers.

1.3 Research Questions and Objectives

The project is guided by the following overarching question: *To what extent can a high-interaction PostgreSQL honeypot combined with behavior analysis record, identify, and explain attack stages from interaction logs, including when attackers use previously unseen techniques?*

1.3.1 RQ1: Logging and Session Reconstruction

RQ1. Which PostgreSQL honeypot logging configurations miss or mis-reconstruct which types of attack actions and sessions, to what extent, and which configuration is sufficient for subsequent behavior analysis?

RO1.1: Measure action-level recall for each attack and benign behavior class across four predefined logging profiles, whose settings are given in the Scope and Limitations section.

RO1.2: Assess whether parsed logs contain the six required session attributes—user, source IP address, database, query, result, and parameter—and evaluate session reconstruction using pairwise precision, recall, and F1 over inactivity thresholds of 1, 5, 15, and 30 minutes.

RO1.3: Select the least costly logging profile that satisfies the pre-established requirements for supporting the RQ2 analysis, considering capture coverage, field completeness, log volume, and query latency.

1.3.2 RQ2: Classification and Evidence

RQ2. Between a rule-based method and a locally run few-shot large language model, which more accurately assigns an attack stage to each query when preceding query context is available,

and how closely does each method's selected evidence match the known evidence?

RO2.1: Compare classification performance and technical output reliability using macro-averaged F1, per-class F1, a confusion matrix, the invalid-output rate (outputs that violate the required JSON schema), and—for the language-model condition—the rate of fabricated evidence identifiers, meaning identifiers that do not correspond to any query in the window.

RO2.2: Evaluate predicted query evidence against dependency-derived ground truth using evidence precision, recall, and F1, and manually review a preselected 20% sample to assess ground-truth quality.

RO2.3: Compare context windows containing the current query and 0, 2, or 4 preceding queries, measuring classification, evidence quality where context is available, and inference time.

1.3.3 RQ3: Generalization to Unseen Technique Families

RQ3. How much do the classification and evidence-matching performance of the two methods decrease on attack-technique families excluded from method development, compared with variations of known families?

RO3.1: Measure the change in macro-averaged F1 and evidence F1 between the known-family evaluation set and the isolated unseen-family set.

RO3.2: Analyze per-class confusions and changes in fabricated evidence identifiers on unseen families.

RO3.3: Examine how SQL structural novelty, PostgreSQL-specific functions or extensions, and the complexity of inter-query dependencies relate to performance decline.

The attack labels are a project-defined taxonomy aligned with relevant MITRE ATT&CK concepts; they are not presented as a verbatim set of official MITRE ATT&CK tactics. Each scenario step will carry an explicit stage label and dependency links so that action, session, and evidence ground truth can be generated consistently.

1.4 Significance of the Project

The project has methodological and practical significance. Methodologically, it evaluates the entire path from database interaction to an interpretable query-level result. It separates logging

coverage from session reconstruction, and separates label accuracy from evidence quality. This makes it possible to identify whether an analysis error originates in missing telemetry, incorrect session grouping, stage classification, or evidence selection. The use of scenario-family grouping and isolated held-out families is intended to reduce query-level leakage and provide a more meaningful test of generalization.

Practically, the comparison can inform the choice of a logging profile by making coverage, field completeness, storage, and latency trade-offs explicit. The local deployment and synthetic sensitive-looking data are designed to support repeatable experiments without paid inference services or exposure of real organizational data. A common parser schema and reproducible scenario generator can also support future extension to additional PostgreSQL behaviors, classifiers, or logging profiles. The results should be interpreted as evidence about the controlled test environment and not as a direct estimate of detection rates in production networks.

1.5 Scope and Limitations

The project is limited to a locally run PostgreSQL honeypot in Docker, populated with fabricated data and sensitive-looking decoy tables. The environment is intended for controlled experimentation and is not designed to expose an unrestricted service to the public Internet. A parameterized Python scenario generator will vary query structure, object names, order, client or tool characteristics, and timing jitter. Scenarios will be informed by attack behaviors reported in the reviewed literature, including credential guessing, post-authentication data access, and program-execution features reported by Song et al. [1] and post-exploitation credential discovery described by Gómez-López et al. [5]. Their exact query sequences and dependency links will be documented as synthetic ground truth rather than as replicas of observed campaigns.

Logging profiles

RQ1 compares four logging profiles, with the exact parameter values frozen in the methodology before scenario runs. C0 sets `log_statement = all`. C1 enables pgAudit session logging. C2 extends C1 with `log_connections`, `log_disconnections`, `log_min_error_statement`, and `pgaudit.log_parameter`. C3, a proxy or packet-capture configuration, is conditional on the time available and will be reported only if implemented. Parsed records will be normalized to a shared schema. Sessionization will use source IP, user, and inactivity thresholds, and evaluation

will compare reconstructed grouping with scenario ground truth. The experiment will record action recall, field completeness, pairwise session metrics, log volume, and query latency.

Taxonomy and labeling

The seven labels are a project taxonomy: six attack stages and one benign class, Noise. Statements that invoke program-execution features, such as `COPY ... FROM PROGRAM`, are labeled Impact, because they change system state by running operating-system commands and do not fit any other class; this mapping is a documented PostgreSQL attack pattern [1]. Labeling rules for file-access and large-object functions such as `pg_read_file` and `lo_export` will be written before scenario generation, and any ambiguous case will be recorded in the ground-truth documentation.

Classification and evidence

RQ2 will compare a rule-based classifier with a locally hosted language model of about seven to eight billion parameters, served through Ollama. Candidate models include Qwen2.5-7B-Instruct, Llama-3.1-8B-Instruct, and Mistral-7B-Instruct; the final choice will be made by a speed test on about twenty windows. Both methods will receive consistently normalized query windows. The model will produce JSON containing a label and a list of evidence identifiers. The schema constrains the format but does not restrict identifiers to the window at decoding time, so that fabricated identifiers can be measured rather than forced to zero. Development and evaluation will be split by session or scenario family, and the test set will remain sealed until the methods are fixed. Evidence ground truth will consist of the target query and its declared dependencies that remain within the context window. Smaller models were reported to perform weakly on comparable alert-interpretation tasks [3], so the project will treat weak performance as a possible outcome rather than an assumption to be avoided. Output sensitivity to evaluation-set size, observed for flow-level LLM classification [6], will be checked by reporting results at more than one evaluation size.

RQ3 will use two or three attack-technique families isolated from the rule and prompt authors and will not permit post hoc tuning on those families. The main limitations are the finite and synthetic nature of the scenarios, the dependence of ground-truth quality on scenario design, and the possibility that behavior in a local emulation differs from interaction with a production database. Decoy realism is also a limitation: prior work shows that an advanced attacker can

fingerprint most common open-source honeytokens without triggering an alert [7], so the synthetic records will be checked for plausibility but are not assumed to be undetectable. A small number of held-out technique families cannot represent every unknown attack. Model outputs may also vary with hardware, model version, and inference settings; these will be recorded for reproducibility. Any manual evidence review will be limited to the prespecified sample and does not replace broader independent annotation.

1.6 Report Organization

This course project report introduces the research problem, objectives, significance, and scope. The planned project work is organized around three research questions: RQ1 evaluates logging coverage and session reconstruction; RQ2 compares rule-based and few-shot language-model classification and evidence selection; and RQ3 tests generalization to isolated attack-technique families. Subsequent project submissions will document the implementation and methodology, experimental results, discussion, and conclusions as those stages are completed.

Chapter 2

Literature Review

2.1 Scope of the Review

This review examines research that supports the main stages of the proposed project: database honeypot deployment and attacker behavior, audit logging and attack reconstruction, learning-based interpretation of security data, generalization to unseen attack behavior, and the evaluation of honeypot realism. The reviewed studies differ in platform, interaction level, data type, and evaluation setting. Their results therefore inform the design of the project, but they do not predict performance on its controlled PostgreSQL query traces. Where a source was available only as a title or a short abstract, the description below is limited to what that source states.

2.2 Database Honeypots and Attacker Behavior

Song, Smaragdakis, and Griffioen deployed 278 honeypots over 20 days in March–April 2024 [1]. Their 220 low-interaction honeypots emulated MySQL, MSSQL, PostgreSQL, and Redis and revealed relatively low scanning activity but persistent brute-force attempts. Their 58 medium- and high-interaction honeypots revealed three types of exploitation: direct manipulation of the database, ransom-driven copying and deletion of data, and use of the DBMS as a route to take over the underlying system. This is the most directly relevant empirical study for the project, because it covers PostgreSQL and multi-step behavior. Its deployment was Internet-facing and differs from the local controlled experiment planned here, so its observed volumes and frequencies are not treated as expected project results.

Rocha and Pereira used honeypot tools to characterize attacks on relational DBMSs and reported that the MySQL DBMS was the most targeted system among those analyzed [8]. Their study examines MySQL, PostgreSQL, and Oracle decoys, but its target prevalence results are specific to their deployment and do not transfer to PostgreSQL in the present setting. Its value

for this project is as motivation for DBMS-specific measurement.

Wegerer and Tjoa survey database honeypots and honeytokens and describe a MySQL honeypot built with open-source components, including an audit plugin that records queries and login attempts [2]. The work shows that database audit mechanisms are a central design concern for a database honeypot, although its implementation is MySQL-specific and its audit plugin cannot be assumed to apply unchanged to PostgreSQL.

Work focused on PostgreSQL and MS SQL honeypots treats post-authentication interaction as the object of capture. Al Hafidz et al. study enhancing detection in a PostgreSQL shell honeypot through interactive query responses [9]. Karo Karo et al. study capturing post-login behavior in MS SQL honeypots through a protocol-aware TDS relay [10]. Both titles indicate that what happens after authentication is the central capture problem, which is the concern addressed by RQ1. Their full evaluation results were not available for this review, so no quantitative comparison is drawn from them.

Gómez-López et al. present SQLiAgent, a multi-agent framework for SQL injection auditing whose LLM-assisted post-exploitation mode interprets database schemas, identifies credential tables, and generates privilege-escalation SQL statements [5]. The work is offensive and is validated on the OWASP Broken Web Applications testbed, but it documents the behaviors that the project's Credential Access and Privilege Escalation scenarios must represent. It also shows that LLM-assisted tools can produce such statements from inferred schema structure, which is relevant to the threat model.

2.3 Audit Logging, Session Analysis, and Attack Reconstruction

Database audit trails provide structured records that can support analysis beyond individual query inspection. Zimniak, Getta, and Benn transform database audit-trail information to discover periodic patterns in workload activity [11]. Their work concerns workload prediction rather than security detection, but it demonstrates the value of preserving ordering and timing when analyzing sequences of database operations.

Biswas and Niemi examine how auditing requirements determine which database logs must be collected [12]. They identify a practical trade-off: insufficient logging prevents reliable auditing, while redundant logging adds cost and overhead on database performance. This motivates the project's comparison of logging profiles using action capture, required-field completeness, log volume, and query latency. Zeng, Xiao, and Chen similarly study auditing overhead

and adaptation in the Linux audit framework and propose adaptive logging that changes the event types and frequencies collected [13]. Although their work is not specific to PostgreSQL, it supports treating instrumentation overhead as part of the evaluation rather than assuming that more detailed logging is cost-free.

Choi and Lee analyze the internal structure of the Microsoft SQL Server transaction log in unallocated disk space [14]. Their analysis indicates that INSERT, UPDATE, and DELETE queries correspond one-to-one with log records, whereas CREATE and DROP TABLE are represented by combinations of several records within a single transaction. For this project the finding is a warning for parser design: a single query may produce several log records, and some statements may produce none at all, so the parser must not assume a fixed correspondence between queries and records.

Goseva-Popstojanova and colleagues characterize malicious web traffic captured by high-interaction honeypots that ran fully functional three-tier web systems [15]. They describe malicious sessions with 43 session attributes, including duration, number of requests, and bytes transferred, and show that supervised learners separated attack sessions from vulnerability-scan sessions with high detection and low false-alarm rates. Their use of session-level attributes motivates the session features and session-level evaluation in RQ1, although their domain is HTTP rather than database queries.

Landauer et al. introduce CAM-LDS, a labeled log dataset of seven attack scenarios covering 81 techniques across 13 tactics, collected in a fully open-source and reproducible environment [16]. Their illustrative study uses an LLM to interpret the logs, and reports that correct attack techniques were predicted perfectly for about one third of attack steps and adequately for another third. The result suggests that LLM-based log interpretation is partially successful and that a substantial share of steps remains difficult, which supports reporting per-class results and not only aggregate scores in this project.

SLEUTH reconstructs attack scenarios in real time from commercial off-the-shelf audit data using a main-memory dependency graph and tag-based techniques for source identification and impact analysis [17]. TraceBack likewise reconstructs attack scenarios from provenance graphs, using causal analysis and sequence modeling to reduce dependency explosion and false positives [18]. These systems operate on host and provenance data rather than database query traces, but they support modeling events as dependent steps. Zambianco et al. further show that using execution preconditions and effects of ATT&CK techniques to model attack paths allows decoy

selection to account for the logical dependencies between attack steps [19]. This provides the conceptual basis for the depends_on annotations used to generate evidence ground truth.

2.4 LLM-Based Interpretation and Technique Mapping

Schärmer et al. evaluate LLMs for interpreting IDS alerts, including distinguishing attack-related alerts from false positives and associating alerts with attack techniques [3]. They examine how additional logs, configuration data, and few-shot examples affect classification performance and consistency, and conclude that carefully chosen examples and system context support effective classification. Their inputs are network and host alerts rather than SQL statements, so the findings must be validated on query data before they are assumed to apply here.

Meng et al. present RHINO, which maps network logs to adversarial tactics and techniques through guided reasoning [4]. Their abstract reports that existing LLM-based methods frequently hallucinate technique names or produce decontextualized mappings because they classify in a single step, and that these failures motivate a reasoning-based approach. Meng et al. also present CYBERTEAM, a benchmark that organizes threat hunting into a standardized workflow of 30 tasks and 9 operational modules, and report that this structure improves results over open-ended reasoning strategies such as in-context learning, chain-of-thought, and tree-of-thought prompting [20]. Both works motivate testing whether structured context improves query-level classification and whether the selected evidence is actually supported by logged events.

Yang et al. propose APTREC, which uses an LLM to recognize APT tactics and techniques from attack subgraphs through a two-stage architecture: a technique recognition module and a tactic reasoning module [21]. They report a 13% F1-score improvement and a 9% recall increase over representative existing methods, together with explainability. Its two-stage design is a useful contrast to the single-step prompt planned for RQ2, although its subgraph input differs from the query windows used here.

Wang et al. study zero-shot multi-label identification of attack techniques in which LLM reasoning is augmented with semantic evidence discovery [22]. Its relevance to RQ2 lies in the explicit linkage between predicted labels and supporting evidence. The abstract was not available for this review, so the description is limited to the title.

Neves et al. compare zero-shot LLMs with rule-based methods for clinical phenotyping from synthetic patient summaries [23]. GPT-4o achieved a recall of 0.97 and a macro-F1 of

0.92, whereas the rule-based method was highly precise (0.92) but had a recall of only 0.36. The authors also show that combining rule-based and LLM methods can direct manual review to discordant cases. This is the closest published analogue to the RQ2 design: it compares the two approaches on a structured classification task, and its result that rules and LLMs fail in complementary ways suggests that the rule-based and LLM outputs should be analyzed jointly rather than only ranked.

Duan et al. survey few-shot learning for intrusion detection and discuss its potential where labeled examples are scarce, together with open challenges in applying it reliably [24]. The proposed project uses few-shot prompting of a fixed local model rather than training a new few-shot learning method, so the survey serves as background on low-example security classification rather than as a method to replicate.

2.5 Classical Machine Learning and LLMs on Structured Security Data

Liu and Lang survey machine-learning and deep-learning methods for intrusion detection, covering taxonomy, algorithms, datasets, and the challenges of detection accuracy, false-alarm rates, and unknown attacks [25]. Alharthi et al. compare classical machine-learning and deep-learning models on binary and multiclass intrusion datasets and find that the strongest model depends on both the dataset and the classification task [26]. These studies support reporting macro-F1 and per-class results and including transparent baselines. Their feature spaces differ from SQL query logs, so their numerical results cannot be transferred to the proposed task.

Rehman et al. serialize network flows into compact text and study zero-shot, instruction-guided, and few-shot prompting with grammar-constrained outputs and a calibrated decision threshold [6]. They find that unguided prompting is unreliable and that instructions plus boolean flags substantially improve detection, while decision quality decreases as the evaluation set grows from 200 to 2000 flows. Tabular baselines remain more stable and faster. Mehavilla et al. benchmark LLMs against machine-learning and deep-learning models on Zeek logs and report that LLMs achieve F1-scores above 95% but do not outperform the best classical model, XGBoost, which reaches 96.96% while using only 4% of available CPU [27]. Together these studies caution against assuming that an LLM is preferable for structured classification, and they motivate a direct comparison under identical inputs and splits.

2.6 Generalization to Unseen Attack Behavior

Generalization is the central concern of RQ3. Lust and Condurache survey methods for assessing whether a deep neural network's output is reliable for a given input, grouping predictive uncertainty, out-of-distribution detection, and adversarial example detection within the broader question of generalization performance [28]. Their framing supports the project's distinction between variations of known technique families and unseen families, and suggests that out-of-distribution behavior should be measured rather than assumed. Zhou and Wang report that transformer networks face difficulty generalizing to out-of-distribution data and propose GROD, which uses synthetic outliers and penalizes out-of-distribution misclassification during training [29]. The method is designed for neural training rather than for prompted LLMs, so it does not directly apply here, but it confirms that transformer generalization to unseen distributions is a recognized problem.

The scarcity of labeled multi-stage data is a related obstacle. Das and Nair publish a dataset of normal smart-home traffic and seven multi-stage attack scenarios, motivated by the observation that multi-stage attacks are more difficult to detect [30]. Their domain is network flows, but the paper illustrates why the project must generate its own labeled, dependency-annotated query scenarios and document the generation process in detail.

2.7 Honeypot Realism, Synthetic Decoy Data, and Evaluation

A honeypot's value depends on the depth and quality of the interactions it sustains. Nunez Migliorisi proposes telemetry-based measures of realism and deception effectiveness, including attacker dwell time, interaction depth, behavioral diversity, and malware family richness, and argues that raw connection counts are insufficient [31]. Ferguson-Walter et al. analyze a controlled experiment with more than 130 professional red teamers and compare measures of forward progress across four conditions defined by the presence of decoys and whether attackers were informed about deception. They report that the combination of decoys and information about deception had the greatest impact on attacker behavior relative to a control condition [32]. These works support using multiple indicators of engagement, although the current project primarily evaluates observability and analysis quality in controlled scenarios.

Synthetic decoy values also need validation. Weinberg proposes PHANTOM, which generates honeytokens conditioned on organization-specific naming and value distributions and

evaluates them with a believability score combining syntactic validity, semantic coherence, statistical plausibility, and human acceptance [33]. The same decomposition can guide a plausibility check on the synthetic sensitive tables in this project, although the project's records are not honeytokens and the score will not be reproduced in full. Msaad et al. show that an advanced attacker can fingerprint most common open-source honeytokens without triggering an alert [7], which is a limitation the project must acknowledge.

Adaptive and LLM-based deception show the direction of recent work. Kouremetis et al. present Mirage, an emulation and simulation system for testing deceptions against adversaries that use AI search and planning [34]. Wushishi et al. propose D3O-IIoT, a reinforcement-learning approach that coordinates honeypot deployment, moving-target defense, and fake telemetry in industrial IoT networks [35]. Safargalieva et al. present OHRA, an LLM-driven honeypot that supports multiple protocols, with SSH, Telnet, and HTTP fully interactive and FTP, SMTP, IPP, and SNMP partially implemented [36]. Bridges et al. systematize research on honeypots and LLMs, providing a taxonomy of detection vectors, a canonical architecture for LLM-powered honeypots, an evaluation framework, and a discussion of log analysis as a route to automated intelligence [37]. Their review highlights the shortage of realistic labeled data and the risk that intelligent adversaries detect LLM-driven behavior. The present project uses an LLM to analyze recorded query traces rather than to generate responses, so response fidelity is not an outcome measure, but the review supports explicit evaluation criteria and cautions against treating LLM output as self-validating evidence.

2.8 Summary of the Literature Review

The reviewed database honeypot studies show that DBMS-specific decoys can expose persistent authentication attempts and deeper multi-step behavior, and that the interaction level and the capture mechanism determine what can be observed [1], [9], [10]. Audit-trail and reconstruction studies show why event order, timing, dependencies, and field coverage matter, while also identifying logging overhead as a relevant cost [12], [13], [14]. Research on LLMs for security data provides useful baselines, contextual prompting strategies, evidence-oriented designs, and evaluation practices, but its results on network flows, alerts, and clinical text do not establish performance on SQL query traces [3], [4], [6], [23]. Generalization studies motivate isolating unseen families rather than relying on random splits [28], and honeypot evaluation studies show that engagement and realism require more than counting connections [31], [32].

A gap remains at the intersection of these areas. The reviewed sources do not establish which PostgreSQL logging profile is sufficient to reconstruct labeled query sessions in a controlled honeypot, nor whether a local few-shot LLM improves query-stage classification and evidence selection over a rule-based method when evaluated on technique families withheld from development. This project addresses that gap by evaluating capture coverage, session reconstruction, classification, evidence quality, and generalization in a single reproducible pipeline. The resulting conclusions will apply to the tested configuration and scenarios, not directly to all production databases or real-world attackers.

2.9 Contribution of Research

The planned contribution is an integrated and reproducible evaluation of a local PostgreSQL honeypot and its behavior-analysis pipeline. Specifically, the project will:

C1: Compare four predefined logging profiles using action-level capture recall, completeness of required event fields, log volume, and query latency.

C2: Measure query-session reconstruction across inactivity thresholds against labeled scenarios with explicit dependencies between attack steps.

C3: Compare a rule-based classifier with a locally hosted few-shot LLM on the same query windows, reporting macro-F1, per-class performance, invalid outputs, fabricated evidence identifiers, and evidence precision, recall, and F1.

C4: Test generalization on isolated attack-technique families that are excluded from rule and prompt development.

These are planned research contributions, not reported experimental findings. The project will use the results to identify the logging and analysis trade-offs observed in its controlled environment and to provide a basis for future extensions.

Chapter 3

Methodology

3.1 Research Design

The study is a quantitative, controlled-environment evaluation organized into three experiments, each answering one research question. RQ1 measures how well four PostgreSQL logging profiles capture attack actions and reconstruct sessions. RQ2 compares a rule-based classifier with a locally hosted few-shot large language model (LLM) on query-level attack-stage labels and evidence selection. RQ3 measures how much the performance of both methods declines when they face attack-technique families that were excluded from their development.

All three experiments share one pipeline so that differences between results can be attributed to the variable under study rather than to inconsistent data handling. The pipeline has six stages: (1) a parameterized scenario generator produces attack and benign query sequences together with ground-truth labels and dependencies; (2) the sequences are executed against a local PostgreSQL honeypot under a chosen logging profile; (3) the resulting logs are parsed into a common event schema; (4) for RQ1, parsed events are grouped into sessions; (5) for RQ2 and RQ3, the events are arranged into query windows and classified; (6) predictions are compared with the ground truth using the metrics defined in Section 3.6.4.

3.2 Honeypot Environment and Logging Profiles

The honeypot is a PostgreSQL server running in Docker on a local machine. It is not exposed to any public network. Its databases contain fabricated data, including decoy tables whose names and contents resemble sensitive records such as user accounts and payment information. No real personal or organizational data is used.

RQ1 compares four logging profiles. Their settings are summarized in Table 3.1. The profiles are ordered by increasing detail, and the expected gaps listed in the table are hypotheses

that RQ1 is designed to test rather than established facts.

Table 3.1: Logging profiles compared in RQ1.

Profile	Configuration	Expected gaps (hypotheses)
C0	`log_statement = all`	Statements that fail to parse are not recorded; failed logins are not recorded; prepared statements are recorded as templates without parameter values.
C1	pgAudit session logging	Connection and disconnection events are not recorded; parameter values of prepared statements are not recorded. Statement types and failed statements are classified.
C2	C1 plus `log_connections`, `log_disconnections`, `log_min_error_statement`, and `pgaudit.log_parameter`	Expected to capture all six session attributes: user, source IP, database, query, result, and parameters.
C3	Proxy or packet capture (conditional)	Included only if time permits; reported separately if implemented.

The six session attributes listed in the last row are the minimum fields required for session reconstruction and for the evidence analysis in RQ2. RQ1 checks, for each profile, whether each attribute is present in the parsed records.

3.3 Attack Taxonomy and Scenario Generation

3.3.1 Behavior labels

Each query is assigned one of seven labels. Six labels describe attack stages and one describes benign activity. The labels are a project taxonomy aligned with the concepts of the MITRE ATT&CK framework; they are not a verbatim copy of its tactics. Table 3.2 defines each label and gives representative PostgreSQL statements.

Statements that invoke program-execution features, such as `COPY ... FROM PROGRAM`, are assigned to Impact. They change system state by running operating-system commands, and no other label fits them. This rule is fixed before scenario generation and applies to every scenario and to the ground-truth files.

Labeling rules for file-access and large-object functions, such as `pg_read_file` and `lo_export`, will be written before scenario generation. Any statement that the rules cannot assign unam-

Table 3.2: Behavior labels and representative statements.

Label	Definition	Representative statements
Discovery	Probing the structure of the database and server	Queries on `information_schema`, `pg_catalog`, `version()`
Credential Access	Obtaining or guessing account credentials	Reading credential tables; repeated failed logins
Privilege Escalation	Gaining higher database privileges	`ALTER USER ... SUPERUSER`, `GRANT`
Collection	Reading sensitive data in bulk	`SELECT` over sensitive tables, large result sets
Exfiltration	Moving data outside the database	`COPY ... TO`, `lo_export`, `dblink` to another server
Impact	Altering or destroying data or structure	`DROP TABLE`, `UPDATE`, `DELETE`, `COPY ... FROM PROGRAM`
Noise	Ordinary queries that are not part of an attack	Routine application-style reads and writes

biguously is recorded in the ground-truth documentation rather than resolved by the classifier.

3.3.2 Parameterized scenario generator

A Python generator produces attack and benign sessions from a set of templates. Each template is a sequence of steps. Every step carries three fields: a stage label from Table 3.2, a unique step identifier, and a `depends_on` list naming the earlier steps whose results the step uses. The `depends_on` links define the evidence ground truth for RQ2. Because the links are declared in the template, the ground truth is generated automatically rather than annotated after the fact.

The generator varies the following parameters to produce distinct instances of each template: table and column names, step order where the template permits it, client tool or library, literal values, and inter-statement timing with random jitter. Step identifiers are kept in the generator's own records and are never embedded in the SQL text, so that the honeypot logs and the classifier input contain no information that a real attacker could not produce.

Scenarios are grouped into families. A family is a set of scenarios that share the same main technique and target. The known families are used to develop the rules and prompts in RQ1 and RQ2. Two or three held-out families are used only in RQ3. These held-out families are

written by a team member who does not write the rules or prompts, and they are fixed before any method is developed, so that they cannot be adapted to after the fact. The benign Noise sessions are generated from routine query patterns and are mixed into every experiment so that the classifiers must distinguish attacks from ordinary activity.

3.4 Data Collection

3.4.1 RQ1 data

Every scenario variant is executed under each of the four logging profiles. The same scenario set is therefore observed four times, which allows coverage to be compared directly. The scenario runner records, on the client side, every statement it sends together with its send timestamp, connection identifier, and step identifier. These client-side records provide the ground truth for matching each honeypot log record to a scenario step, without modifying the SQL text.

For each profile, the parser converts the raw PostgreSQL log into the common schema. The schema has one record per event, with fields for timestamp, user, source IP, database, statement text, result status, and bound parameters. Fields that a profile does not record are stored as missing rather than inferred.

Overhead is measured in the same runs. Log volume is measured as the number of bytes written per session. Query latency is measured as the wall-clock time per statement, with and without each profile enabled, on the same hardware. Each measurement is repeated, and median values with interquartile ranges are reported.

3.4.2 RQ2 and RQ3 data

The classification dataset consists of 100 labeled queries drawn from the known scenario families and the Noise sessions. These queries are collected under profile C2, which records all six attributes. Classification is therefore not limited by missing fields. If RQ1 selects a lighter profile, RQ2 will be repeated on that profile as a sensitivity check, and any difference will be reported.

Before classification, every query is normalized. Literal values are replaced with typed placeholders, such as `<STR>` and `<NUM>`. Table and column identifiers are renamed according to a consistent mapping that is generated independently for each scenario, so that a method cannot learn a specific table name as a cue for a label. The original statement text is retained in the

ground-truth files, and the normalized text is used only as the classifier input.

3.5 Data Splitting

The dataset is split with `StratifiedGroupKFold` from scikit-learn [38]. The configuration is `n_splits = 3`, `shuffle = True`, and `random_state = 42`. The group variable is the session or scenario identifier, so that all queries from the same session are kept in the same fold. This preserves the `depends_on` chains that RQ2 relies on and prevents query-level leakage between training and test data. The stratification variable is the label, which keeps the distribution of the seven classes approximately equal across folds, so that macro-averaged F1 reflects all classes.

One fold, containing about one third of the queries, is held out as the sealed test set. The remaining two folds, about two thirds of the data, are used for development and training. Within this portion, a further grouped split is used for tuning. The test fold is not used to select rules, prompt examples, or context length, and it is evaluated once after those choices are fixed.

Note on split proportions: This configuration gives approximately 67% training and 33% test data, not the 70% and 30% originally planned. The 3-fold configuration is kept because `StratifiedGroupKFold` generates folds of this size. With 100 queries, the test fold contains about 33 queries across seven classes, so some classes will have very few test examples. Per-class results on the test fold must be interpreted with this limitation in mind (see Section 3.7).

3.6 Analysis Techniques

3.6.1 RQ1: coverage and session reconstruction

Action-level recall is computed for each label group g and each profile p:

$$\text{Recall}_{p,g} = \frac{\text{number of actions in } g \text{ detected under } p}{\text{number of actions in } g}.$$

An action is detected if a parsed record matches its executed statement after normalization and contains every attribute that its label requires. For example, a failed-login action requires the user, the source IP, and the result status. An action that appears in the log but lacks a required attribute is counted as not detected and is reported separately as a field-completeness failure.

Field completeness is the proportion of parsed records that contain all six session attributes under each profile.

Sessions are reconstructed by grouping parsed events that share the same source IP, user, and database, with a new session starting whenever the gap between consecutive events exceeds the inactivity threshold. Thresholds of 1, 5, 15, and 30 minutes are tested. A reconstructed session is counted as matched to a true session if the two contain exactly the same set of actions. Session recall is the number of matched sessions divided by the number of true sessions. Session precision is the number of matched sessions divided by the number of reconstructed sessions. Session F1 is their harmonic mean. Pairwise precision, recall, and F1 over event pairs are reported as a secondary measure, because a single merged or split session affects the exact-match score strongly.

The selection rule for RQ1 has three parts. A profile is eligible only if it reaches a recall of at least 95% for every label group and contains all six attributes for the RQ2 analysis. Among eligible profiles, the one with the lowest combined log volume and latency cost is selected. If no profile is eligible, the best available profile is selected and the shortfall is reported explicitly.

3.6.2 Rule-based classifier

The rule-based method assigns a label to each query by matching conditions over the query window. Each rule may refer to the current query and to earlier queries in the window, so the rules can express dependencies such as a query that reads a value obtained by an earlier query. Rules are written and refined only on the development folds. Their output contains the predicted label and the identifiers of the queries that satisfied the conditions of the matched rule. These identifiers are the rule-based evidence set.

3.6.3 LLM few-shot classifier

The LLM method uses Llama 3.1 8B Instruct served locally through Ollama [39], with no fine-tuning. Temperature is set to 0 and a fixed random seed is used. The model version, Ollama version, and hardware are recorded with every run. The model is selected by a speed test on about 20 windows. If the 8B model is too slow for the planned number of runs, a smaller model in the same family will be used, and the change will be reported.

The prompt is built from five components, following the structure of guided reasoning approaches for security classification [4], [20]. Prompt design also affects reliability substantially

for structured security data [6], which motivates the explicit output constraints below.

1. **Role and task.** The model is instructed to act as a cybersecurity analyst who classifies PostgreSQL statements into attack stages.
2. **Label space.** The model may output only one of the seven labels in Table 3.2.
3. **Output format.** The model must return only a JSON object of the form `{"label": ..., "evidence_ids": [...]}`, with no explanation outside the object.
4. **Few-shot examples.** Two or three input–output pairs are included. They are drawn only from the development folds and never from the test fold.
5. **Target input.** The query window, containing the current query and the k preceding queries from the same session, is given as a list of objects with identifiers.

An example of the prompt structure is shown below. The labels and the example are placeholders for the final prompt text.

```
You are a cybersecurity analyst. Classify the last query in the window
into exactly one of: Discovery, Credential Access, Privilege Escalation,
Collection, Exfiltration, Impact, Noise.
Return only JSON: {"label": "<label>", "evidence_ids": ["<id>", ...]}
No text outside the JSON.

Example 1:
Input: [{"id":"q1","query":"SELECT * FROM users"},
        {"id":"q2","query":"COPY users TO '/tmp/data.csv'"}]
Output: {"label": "Exfiltration", "evidence_ids": ["q1"]}

Input: <query window inserted here>
```

Output is parsed with `json.loads` inside a `try`/`except` block. An output that cannot be parsed, or that contains a label outside the seven allowed values, is counted as an invalid output and as an incorrect prediction. No retry is performed, so that the invalid-output rate reflects the first response of the model.

Evidence identifiers are not restricted to the window at decoding time. This allows fabricated identifiers to be measured rather than prevented by construction. An identifier is counted as fabricated if it does not correspond to any query in the window.

3.6.4 Evaluation metrics

Classification performance is reported as macro-averaged F1, which is the unweighted mean of the per-class F1 scores. This prevents the frequent classes from dominating the result. Per-class precision, recall, and F1 are reported alongside, together with a confusion matrix that shows which label pairs are confused. For example, the matrix shows whether Credential Access queries are most often predicted as Discovery.

Technical output reliability is measured by the invalid-output rate and by the fabricated-evidence rate, defined as the number of fabricated identifiers divided by the total number of identifiers returned by the LLM.

Evidence quality is measured against the ground truth derived from `depends_on`. For a target query t in a window, the ground-truth evidence set is $E_t^{\text{gt}} = \{t\} \cup \{d \in \text{depends_on}(t) : d \text{ lies within the window}\}$. The predicted evidence set E_t^{pred} is the set of identifiers returned by the method. Two measures are reported: the Jaccard index $|E_t^{\text{pred}} \cap E_t^{\text{gt}}|/|E_t^{\text{pred}} \cup E_t^{\text{gt}}|$, averaged over all queries, and evidence precision, recall, and F1 computed over the same sets. Evidence is evaluated only for $k \geq 2$, because with $k = 0$ the window contains only the target query, and evidence selection is then trivial.

Ground-truth quality is checked by manual review of a 20% sample of the queries, stratified across labels. Each sampled evidence set is rated by a reviewer who did not write the rules or the prompts as complete (the automatically generated set is correct), missing a necessary query (incomplete), or containing a superfluous query (over-inclusive). The Jaccard index between the automatic and manual sets is reported as an estimate of ground-truth reliability. The manual review is performed only after the rules and prompts are fixed, to avoid tuning the ground truth toward a method.

3.6.5 Effect of context length

The context length $k \in \{0, 2, 4\}$ is varied while everything else is held constant. For each value, macro-F1, evidence F1 (for $k \geq 2$), and average inference time per window are recorded. The results are plotted against k to show whether additional context improves classification, whether

it improves evidence selection, and whether the time cost grows faster than the benefit. The value of k that gives the best development performance is carried forward to the test fold.

3.6.6 Uncertainty and comparison

Confidence intervals for macro-F1 and evidence F1 are estimated by bootstrap resampling over sessions rather than over individual queries, because queries within a session are dependent. The two methods are compared on the same test fold, and differences are reported with their bootstrap intervals.

3.6.7 RQ3: performance under unseen families

For each method, the performance drop is defined as $\Delta F_1 = F_1^{\text{known}} - F_1^{\text{new}}$, where F_1^{known} is measured on the known-family test fold and F_1^{new} on the held-out families. A small ΔF_1 indicates that the method generalizes well. The same definition is applied to macro-F1 and to evidence F1. Per-class F1 on the held-out families and the change in fabricated-evidence rate are reported in addition.

To explain the drop, each held-out family is scored on three factors using the rubric in Table 3.3. Each factor is scored from 1 to 3. Two independent coders score each family, and disagreements are resolved by discussion. With only two or three held-out families, the relation between the factor scores and ΔF_1 is descriptive and cannot establish a general effect. It is reported as an observation about these families only.

Table 3.3: Rubric for characterizing the novelty of a held-out attack-technique family.

Factor	1	2	3
SQL structural novelty	Differs only in table names, parameters, columns, or conditions	Nested statements or unusual combinations of logical conditions	Multi-layer or second-order constructions, such as string-built statements
PostgreSQL-specific functions	Standard SQL only	Some system-information functions	Specialized functions or advanced extensions
Dependency complexity	Independent steps or chains with $k = 1$	Chains with $k = 2$ to 3	Long-range, fragmented chains where the relevant steps are far apart

3.7 Limitations of the Methodology

The methodology has several limitations that affect how the results can be interpreted.

First, the dataset is small. With 100 queries, the test fold contains about 33 queries across seven labels, so per-class estimates are unstable and some classes may have only one or two test examples. Results should be reported with bootstrap intervals and interpreted as indicative rather than definitive.

Second, the attack behavior is synthetic. Scenarios are generated from templates informed by the literature [1], [5], [16]. They represent the behaviors the team chose to simulate, not the distribution of real attacks, and the ground truth reflects the template design. Realistic attacks may use techniques outside these templates.

Third, the taxonomy is a project choice. Seven labels cannot represent every PostgreSQL behavior. Boundary cases, such as program-execution statements mapped to Impact, are a labeling decision that may not match other taxonomies, and the mapping rule is applied consistently but may affect the Impact class results.

Fourth, the held-out evaluation is limited. Two or three families can indicate whether performance drops, but they cannot characterize all unknown attack techniques, and the relation between novelty factors and performance drop is not statistically generalizable.

Fifth, the LLM is not deterministic in practice, even with temperature 0 and a fixed seed, because inference libraries and hardware can introduce small numerical differences. Results are therefore reported with the model version, software versions, and hardware specification, and the study does not claim that identical results will be produced on other systems.

Sixth, latency and log volume depend on the hardware and the workload, and the overhead measurements are valid only for the environment in which they are taken.

Finally, the rubric scores and the manual evidence review involve human judgment. Inter-coder agreement is reported to show how consistent these judgments are, but disagreements remain a source of uncertainty.

3.8 Reproducibility and Ethical Considerations

All generator code, parser code, prompts, rule definitions, random seeds, and split indices will be stored in a version-controlled repository so that every table and figure can be regenerated. The honeypot holds fabricated data only, is not exposed to external networks, and does not

run real malware. The scenario generator produces SQL statements only, and no live attack infrastructure is used.

References

[1] Y. Song, G. Smaragdakis, and H. Griffioen, "Decoy databases: Analyzing attacks on public facing databases," in *Proceedings of the 2025 ACM Internet Measurement Conference*, Madison WI USA: ACM, Oct. 28, 2025, pp. 709–726, ISBN: 979-8-4007-1860-1. DOI: 10.1145/3730567.3764481 Accessed: Sep. 25, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3730567.3764481

[2] M. Wegerer and S. Tjoa, "Defeating the database adversary using deception - a MySQL database honeypot," in *2016 International Conference on Software Security and Assurance (ICSSA)*, St. Pölten, Austria: IEEE, Aug. 2016, pp. 6–10, ISBN: 978-1-5090-4388-0. DOI: 10.1109/ICSSA.2016.8 Accessed: Sep. 25, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/7861643/

[3] A. Schärmer, M. Landauer, F. Skopik, M. Wurzenberger, and M. Squarcina, "Let the alerts speak: LLM-based IDS alert interpretation for SOC triage," in *Availability, Reliability and Security. ARES 2026 International Workshops*, ser. Lecture Notes in Computer Science, Z. Afzal, P. Kochberger, B. Atli, and M. Asplund, Eds., vol. 16901, Cham: Springer Nature Switzerland, 2027, pp. 346–364, ISBN: 978-3-032-35578-2 978-3-032-35579-9. DOI: 10.1007/978-3-032-35579-9_19 Accessed: Oct. 27, 2026. [Online]. Available: https://link.springer.com/10.1007/978-3-032-35579-9_19

[4] F. Meng, J. Gui, Y. Li, and Y. Wu, *RHINO: Guided reasoning for mapping network logs to adversarial tactics and techniques with large language models*, Oct. 16, 2025. DOI: 10.48550/arXiv.2510.14233 arXiv: 2510.14233[cs.CR]. Accessed: Oct. 27, 2026. [Online]. Available: http://arxiv.org/abs/2510.14233

[5] J. Gómez-López, D. Penco-Hernández, Ó. D. Gómez-López, F. J. Martínez-López, and N. Padilla-Soriano, "A multi-agent framework for SQL injection auditing with LLM-driven post-exploitation and privilege escalation," *Computers*, vol. 15, no. 10, p. 658,

Sep. 29, 2026, ISSN: 2073-431X. DOI: 10.3390/computers15100658 Accessed: Oct. 27, 2026. [Online]. Available: https://www.mdpi.com/2073-431X/15/10/658

[6] M. A. Rehman, S. I. A. Shah, A. Anwar, and N. Islam, *From flows to words: Can zero-/few-shot LLMs detect network intrusions? a grammar-constrained, calibrated evaluation on UNSW-NB15*, Oct. 26, 2025. DOI: 10.48550/arXiv.2510.17883 arXiv: 2510.17883[cs.CR]. Accessed: Sep. 25, 2026. [Online]. Available: http://arxiv.org/abs/2510.17883

[7] M. Msaad, S. Srinivasa, M. M. Andersen, D. H. Audran, C. U. Orji, and E. Vasilomanolakis, "Honeysweeper: Towards stealthy honeytoken fingerprinting techniques," in *Secure IT Systems*, H. P. Reiser and M. Kyas, Eds., vol. 13700, Series Title: Lecture Notes in Computer Science, Cham: Springer International Publishing, 2022, pp. 101–119, ISBN: 978-3-031-22294-8 978-3-031-22295-5. DOI: 10.1007/978-3-031-22295-5_6 Accessed: Sep. 25, 2026. [Online]. Available: https://link.springer.com/10.1007/978-3-031-22295-5_6

[8] A. Rocha and F. S. Pereira, "What is the most targeted database? using honeypots to characterize attacks on relational databases," *Procedia Computer Science*, vol. 256, pp. 114–121, 2025, ISSN: 18770509. DOI: 10.1016/j.procs.2025.02.103 Accessed: Sep. 25, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S1877050925004600

[9] J. Al Hafidz, C. Lim, and K. E. Silaen, "Enhancing detection postgresql shell honeypot with interactive query response," in *2025 IEEE 2nd International Conference on Cryptography, Informatics, and Cybersecurity (ICoCICs)*, Depok, Indonesia: IEEE, Oct. 22, 2025, pp. 255–260, ISBN: 979-8-3315-5481-1. DOI: 10.1109/ICoCICs68032.2025.11384049 Accessed: Oct. 27, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11384049/

[10] A. A. Karo Karo, C. Lim, and K. E. Silaen, "Capturing post-login behaviors in MS SQL honeypots: Profiling attackers via a protocol-aware TDS relay," in *2025 15th International Conference on Information & Communication Technology and System (ICTS)*, Denpasar, Indonesia: IEEE, Nov. 12, 2025, pp. 1–6, ISBN: 979-8-3315-6699-9. DOI: 10.1109/ICTS67612.2025.11369480 Accessed: Oct. 27, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11369480/

[11] M. Zimniak, J. R. Getta, and W. Benn, "Predicting database workloads through mining periodic patterns in database audit trails," *Vietnam Journal of Computer Science*, vol. 2, no. 4, pp. 201–211, Nov. 2015, ISSN: 2196-8888, 2196-8896. DOI: 10.1007/s40595-015-0042-0 Accessed: Sep. 25, 2026. [Online]. Available: http://link.springer.com/10.1007/s40595-015-0042-0

[12] D. Biswas and V. Niemi, "Transforming privacy policies to auditing specifications," in *2011 IEEE 13th International Symposium on High-Assurance Systems Engineering*, Boca Raton, FL, USA: IEEE, Nov. 2011, pp. 368–375, ISBN: 978-1-4673-0107-7 978-0-7695-4615-5. DOI: 10.1109/HASE.2011.51 Accessed: Sep. 25, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/6113921/

[13] L. Zeng, Y. Xiao, and H. Chen, "Auditing overhead, auditing adaptation, and benchmark evaluation in linux," *Security and Communication Networks*, vol. 8, no. 18, pp. 3523–3534, Dec. 2015, ISSN: 1939-0114, 1939-0122. DOI: 10.1002/sec.1277 Accessed: Sep. 25, 2026. [Online]. Available: https://onlinelibrary.wiley.com/doi/10.1002/sec.1277

[14] H. Choi and S. Lee, "Forensic analysis of SQL server transaction log in unallocated area of file system," *Forensic Science International: Digital Investigation*, vol. 46, p. 301 605, Oct. 2023, ISSN: 26662817. DOI: 10.1016/j.fsidi.2023.301605 Accessed: Sep. 25, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2666281723001178

[15] K. Goseva-Popstojanova, G. Anastasovski, A. Dimitrijevikj, R. Pantev, and B. Miller, "Characterization and classification of malicious web traffic," *Computers & Security*, vol. 42, pp. 92–115, May 2014, ISSN: 01674048. DOI: 10.1016/j.cose.2014.01.006 Accessed: Sep. 25, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0167404814000170

[16] M. Landauer, W. Hotwagner, T. Boenke, F. Skopik, and M. Wurzenberger, *CAM-LDS: Cyber attack manifestations for automatic interpretation of system logs and security alerts*, Mar. 4, 2026. DOI: 10.48550/arXiv.2603.04186 arXiv: 2603.04186[cs.CR]. Accessed: Oct. 27, 2026. [Online]. Available: http://arxiv.org/abs/2603.04186

[17] N. Hossain, "SLEUTH: Real-time attack scenario reconstruction from COTS audit data,"

[18] T. Zhu□, M. Li□, and Z. Wang□□, "TraceBack: Sequence-enhanced and causality-driven attack scenario reconstruction from provenance graphs,"

[19] M. Zambianco, C. Facchinetti, and D. Siracusa, "A proactive decoy selection scheme for cyber deception using MITRE ATT&CK," *Computers & Security*, vol. 148, p. 104 144, Jan. 2025, ISSN: 01674048. DOI: 10.1016/j.cose.2024.104144 arXiv: 2404.12783[cs.CR]. Accessed: Sep. 25, 2026. [Online]. Available: http://arxiv.org/abs/2404.12783

[20] Y. Meng et al., *Benchmarking LLM-assisted blue teaming via standardized threat hunting*, May 28, 2026. DOI: 10.48550/arXiv.2509.23571 arXiv: 2509.23571[cs.CR]. Accessed: Oct. 27, 2026. [Online]. Available: http://arxiv.org/abs/2509.23571

[21] L. Yang, A. Ye, Y. Liu, W. Lu, and C. Huang, "APTREC: APT tactic/technique recognition based on large language model," *Computer Networks*, vol. 280, p. 112 146, 2026, ISSN: 1389-1286. DOI: https://doi.org/10.1016/j.comnet.2026.112146 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1389128626001581

[22] Z. Wang, Z. Fan, Y. Li, X. Zhang, P. Dai, and Z. Tian, "Toward zero-shot multi-label identification of attack techniques: Enhancing LLM reasoning with semantic evidence discovery," *IEEE Transactions on Knowledge and Data Engineering*, vol. 38, no. 7, pp. 4791–4804, Jul. 2026, ISSN: 1041-4347, 1558-2191, 2326-3865. DOI: 10.1109/TKDE.2026.3682468 Accessed: Oct. 27, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11478373/

[23] B. Neves et al., "Zero-shot learning for clinical phenotyping: Comparing LLMs and rule-based methods," *Computers in Biology and Medicine*, vol. 192, p. 110 181, Jun. 2025, ISSN: 00104825. DOI: 10.1016/j.compbiomed.2025.110181 Accessed: Oct. 27, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0010482525005323

[24] R. Duan, D. Li, Q. Tong, T. Yang, X. Liu, and X. Liu, "A survey of few-shot learning: An effective method for intrusion detection," *Security and Communication Networks*, vol. 2021, Y. Guo, Ed., pp. 1–10, Oct. 31, 2021, ISSN: 1939-0122, 1939-0114. DOI: 10.1155/2021/4259629 Accessed: Sep. 25, 2026. [Online]. Available: https://www.hindawi.com/journals/scn/2021/4259629/

[25] H. Liu and B. Lang, "Machine learning and deep learning methods for intrusion detection systems: A survey," *Applied Sciences*, vol. 9, no. 20, p. 4396, Oct. 17, 2019, ISSN: 2076-3417. DOI: 10.3390/app9204396 Accessed: Sep. 25, 2026. [Online]. Available: https://www.mdpi.com/2076-3417/9/20/4396

[26] A. Alharthi, M. Alaryani, and S. Kaddoura, "A comparative study of machine learning and deep learning models in binary and multiclass classification for intrusion detection systems," *Array*, vol. 26, p. 100 406, Jul. 2025, ISSN: 25900056. DOI: 10.1016/j.array.2025.100406 Accessed: Sep. 25, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2590005625000335

[27] L. Mehavilla, M. Rodríguez, J. García, and Á. Alesanco, "Evaluating large language models effectiveness for flow-based intrusion detection: A comparative study with ML and DL baselines," *Artificial Intelligence Review*, vol. 59, no. 2, p. 50, Jan. 9, 2026, ISSN: 1573-7462. DOI: 10.1007/s10462-025-11432-2 Accessed: Sep. 25, 2026. [Online]. Available: https://link.springer.com/10.1007/s10462-025-11432-2

[28] J. Lust and A. P. Condurache, "A survey on assessing the generalization area of neural networks: Uncertainty, out-of-distribution and adversarial samples," *Neurocomputing*, vol. 706, p. 135 003, 2026, ISSN: 0925-2312. DOI: https://doi.org/10.1016/j.neucom.2026.135003 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S092523122602401X

[29] Y. Zhou and Y. Wang, "GROD: Enhancing generalization of transformer with out-of-distribution detection," 2024.

[30] V. Das and B. B. Nair, "A novel multi-stage attack dataset for smart home intrusion detection," *Data in Brief*, vol. 66, p. 112 770, Jun. 2026, ISSN: 23523409. DOI: 10.1016/j.dib.2026.112770 Accessed: Sep. 25, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2352340926003239

[31] T. Noelia Nunez Migliorisi, "EMPIRICAL TELEMETRY-BASED METRICS FOR EVALUATING HONEYPOT REALISM AND DECEPTION EFFECTIVENESS," *International Journal of Network Security & Its Applications*, vol. 18, no. 1, pp. 01–12, Jan. 28, 2026, ISSN: 09752307. DOI: 10.5121/ijnsa.2026.18101 Accessed: Sep. 25, 2026. [Online]. Available: https://aircconline.com/ijnsa/V18N1/18126ijnsa01.pdf

[32] K. J. Ferguson-Walter and M. M. Major, “Examining the efficacy of decoy-based and psychological cyber deception,”

[33] A. I. Weinberg, *PHANTOM: Polymorphic honeytoken adaptation with narrative-tailored organisational mimicry*, May 4, 2026. DOI: 10.48550/arXiv.2605.02992 arXiv: 2605.02992[cs.CR]. Accessed: Sep. 25, 2026. [Online]. Available: http://arxiv.org/abs/2605.02992

[34] M. Kouremetis et al., “Mirage: Cyber deception against autonomous cyber attacks in emulation and simulation,” *Annals of Telecommunications*, vol. 79, no. 11, pp. 803–817, Dec. 2024, ISSN: 0003-4347, 1958-9395. DOI: 10.1007/s12243-024-01018-4 Accessed: Sep. 25, 2026. [Online]. Available: https://link.springer.com/10.1007/s12243-024-01018-4

[35] U. Wushishi, A. Hussain, M. I. Khalid, N. Hussain, M. Jamjoom, and Z. Ullah, “D3o-IIoT: Deep reinforcement learning-driven dynamic deception orchestration for industrial IoT security,” *Scientific Reports*, vol. 16, no. 1, p. 2389, Dec. 21, 2025, ISSN: 2045-2322. DOI: 10.1038/s41598-025-33426-4 Accessed: Sep. 25, 2026. [Online]. Available: https://www.nature.com/articles/s41598-025-33426-4

[36] A. Safargalieva, A. Rüffer, and E. Vasilomanolakis, “OHRA: Dynamic multi-protocol LLM-based cyber deception,”

[37] R. A. Bridges, T. R. Mitchell, M. Muñoz, and T. Henriksson, *SoK: Honeypots & LLMs, more than the sum of their parts?* Apr. 7, 2026. DOI: 10.48550/arXiv.2510.25939 arXiv: 2510.25939[cs.CR]. Accessed: Oct. 27, 2026. [Online]. Available: http://arxiv.org/abs/2510.25939

[38] scikit-learn developers. “Stratifiedgroupkfold,” Accessed: Oct. 28, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html

[39] Ollama. “Ollama documentation,” Accessed: Oct. 28, 2026. [Online]. Available: https://docs.ollama.com/